

Chapter 10 String



Objectives

- ❑ Constructing a String
- ❑ Obtaining String length and Retrieving Individual Characters
- ❑ Concatenation - Substrings - Comparisons
- ❑ String Conversions
- ❑ Finding a Character or a Substring in a String
- ❑ Conversions between Strings and Arrays
- ❑ Converting Characters and Numeric Values to Strings
- ❑ To use the **String** class to process immutable strings (§ 10.10).
- ❑ To use the **StringBuilder** and **StringBuffer** classes to process mutable strings (§ 10.11).



String Type

- ◆ String is a sequence of characters.
- ◆ The char type represents only one character. To represent a string of characters, use the data type called String.
- ◆ For example, the following code declares message to be a string with the value "Welcome to Java".
- ◆ `String message = "Welcome to Java";`



String Type... cont.

- ◆ The String type is not a primitive type.
- ◆ It is known as a reference type.
- ◆ Any Java class can be used as a reference type for a variable (Scanner, System, String).
- ◆ The variable, declared by a reference type, is known as a reference variable that references an object.



String Type... cont.

✦ `String message` = "Welcome to Java";

Here `message` is a reference variable that references a `string` object with contents Welcome to Java.



Constructing a String

- ◆ To create a string from a string literal, use the syntax:

```
String newString = new String(stringLiteral);
```

- ◆ The argument **stringLiteral** is a sequence of characters enclosed inside double quotes.

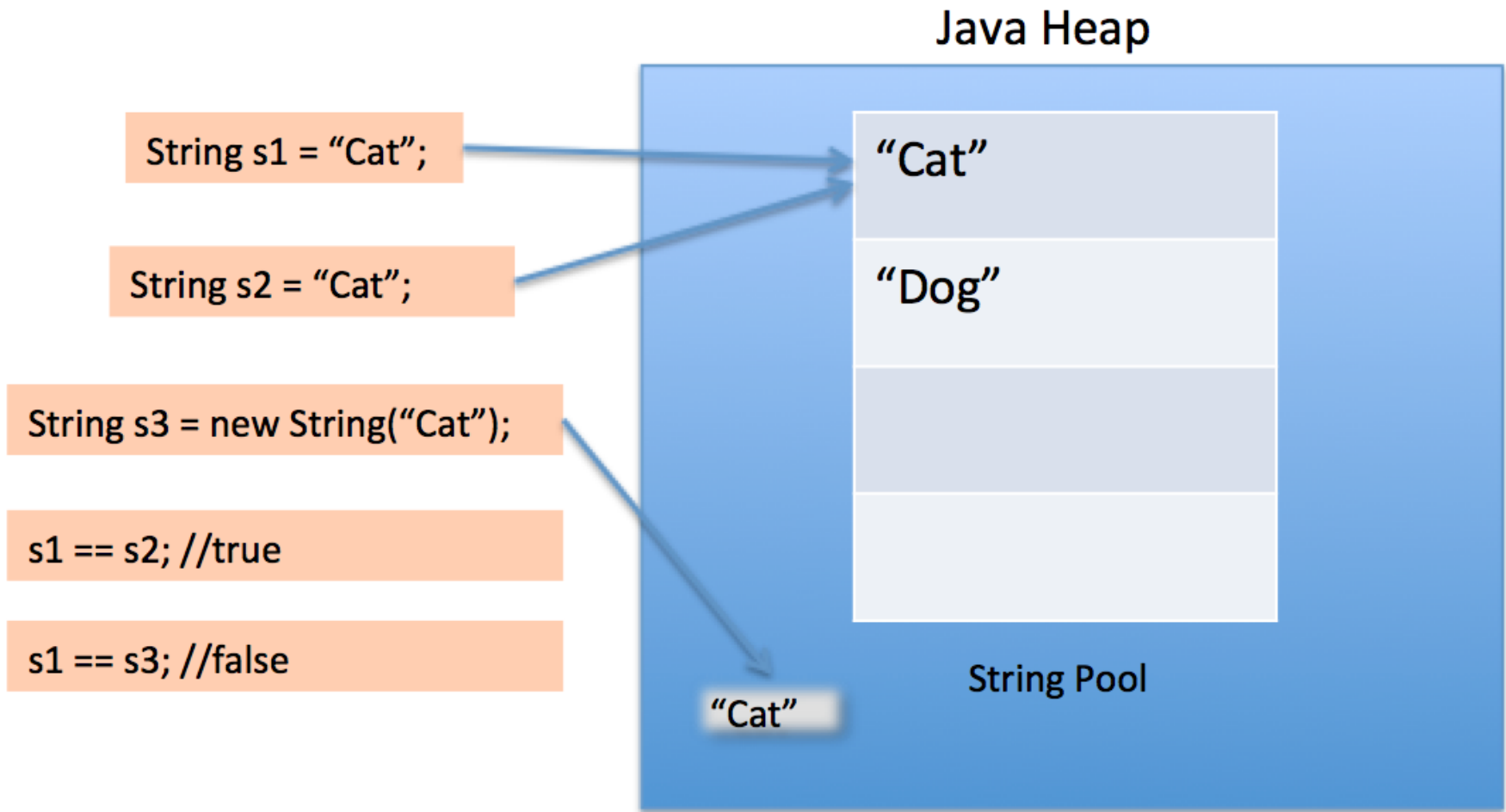
- ◆ The following statement creates a **String** object **message** for the string literal "**Welcome to Java**":

```
String message = new String("Welcome to Java");
```

- ◆ Java treats a string literal as a **String** object. Thus, the following statement is valid:
- ◆ `String message = "Welcome to Java";`



Constructing a String...cont.



Constructing a String...cont.

- ♦ You can also create a string from an array of characters. For example, the following statements create the string **"Good Day"**:

```
char[] charArray = {'G', 'o', 'o', 'd', ' ', 'D', 'a',  
'y'};
```

```
String message = new String(charArray);
```



Simple Methods for String Objects

- ✦ **length()** Returns the **number** of characters in this string.
- ✦ **charAt(index)** Returns the **character** at the specified index from this string.
- ✦ **concat(s1)** Returns a new **string** that concatenates this string with string s1.
- ✦ **toUpperCase()** Returns a new **string** with all letters in uppercase.
- ✦ **toLowerCase()** Returns a new **string** with all letters in lowercase.
- ✦ **trim()** Returns a new **string** with whitespace characters trimmed on both sides.



Comparison Methods for String Objects

- ✦ **equals(s1)** Returns **true** if this string is equal to string s1.
- ✦ **equalsIgnoreCase(s1)** Returns **true** if this string is equal to string s1; it is case insensitive.
- ✦ **compareTo(s1)** Returns an **integer** greater than 0, equal to 0, or less than 0 to indicate whether this string is greater than, equal to, or less than s1.
- ✦ **compareToIgnoreCase(s1)** Same as compareTo except that the comparison is case insensitive.
- ✦ **startsWith(prefix)** Returns **true** if this string starts with the specified prefix.
- ✦ **endsWith(suffix)** Returns **true** if this string ends with the specified suffix.
- ✦ **contains(s1)** Returns **true** if s1 is a substring in this string.

Obtaining substrings for String Object

- ♦ **substring(beginIndex)** Returns this **string**'s substring that begins with the character at the specified beginIndex and extends to the end of the string.
- ♦ **substring(beginIndex, endIndex)** Returns this **string**'s substring that begins at the specified beginIndex and extends to the character at index endIndex – 1, Note that the character at endIndex is not part of the substring.



Finding substrings for String Object

- ♦ **indexOf(ch)** Returns the **index** of the **first** occurrence of ch in the string. Returns -1 if not matched.
- ♦ **indexOf(ch, fromIndex)** Returns the **index** of the **first** occurrence of ch **starting the search at fromIndex** in the string. Returns -1 if not matched.
- ♦ **indexOf(s)** Returns the **index** of the **first** occurrence of string s in this string. Returns -1 if not matched.
- ♦ **indexOf(s, fromIndex)** Returns the **index** of the **first** occurrence of string s in this string **starting the search at fromIndex**. Returns -1 if not matched.



Finding substrings for String Object... cont.

- ◆ **lastIndexOf(ch)** Returns the **index** of the **last** occurrence of **ch** in the string. Returns -1 if not matched.
- ◆ **lastIndexOf(ch, fromIndex)** Returns the **index** of the **last** occurrence of **ch** **starting the search at fromIndex** in this string. Returns -1 if not matched.
- ◆ **lastIndexOf(s)** Returns the **index** of the **last** occurrence of string **s**. Returns -1 if not matched.
- ◆ **lastIndexOf(s, fromIndex)** Returns the **index** of the **last** occurrence of string **s** **starting the search at fromIndex**. Returns -1 if not matched.



Review Question

- ♦ Suppose that s1, s2, and s3 are three strings, given as follows:
- ♦ `String s1 = "Welcome to Java";`
- ♦ `String s2 = "Programming is fun";`
- ♦ `String s3 = "Welcome to Java";`
- ♦ What are the results of the following expressions?
- ♦ a. `s1 == s2`
- ♦ b. `s2 == s3`
- ♦ c. `s1.equals(s2)`
- ♦ d. `s1.equals(s3)`
- ♦ e. `s1.compareTo(s2)`
- ♦ f. `s2.compareTo(s3)`
- ♦ g. `s2.compareTo(s2)`



Answer

♦ a. `s1 == s2`

False

♦ b. `s2 == s3`

False

♦ c. `s1.equals(s2)`

False

♦ d. `s1.equals(s3)`

True

♦ e. `s1.compareTo(s2)`

positive number

♦ f. `s2.compareTo(s3)`

negative number

♦ g. `s2.compareTo(s2)`

0



Review Question

- ♦ Suppose that s1 and s2 are two strings. Which of the following statements or expressions are incorrect?
- ♦ `String s = "Welcome to Java";`
- ♦ `String s3 = s1 + s2;`
- ♦ `String s3 = s1 - s2;`
- ♦ `s1 == s2;`
- ♦ `s1 >= s2;`
- ♦ `s1.compareTo(s2);`
- ♦ `int i = s1.length();`



Answer

String s = "Welcome to Java";

Answer: Correct

String s3 = s1 + s2;

Answer: Correct

String s3 = s1 - s2;

Answer: Incorrect

s1 == s2

Answer: Correct

s1 >= s2

Answer: Incorrect

s1.compareTo(s2);

Answer: Correct

int i = s1.length();

Answer: Correct



Strings Are Immutable

A String object is immutable; its contents cannot be changed.

Does the following code change the contents of the string?

```
String s = "Java";  
s = "HTML";
```

The answer is no

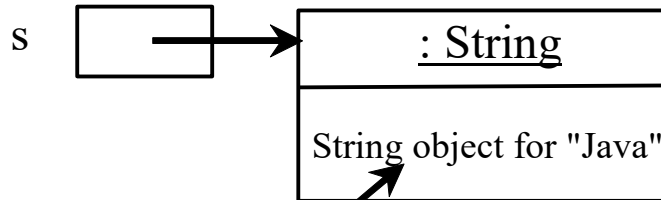


Trace Code

```
String s = "Java";
```

```
s = "HTML";
```

After executing `String s = "Java";`



Contents cannot be changed

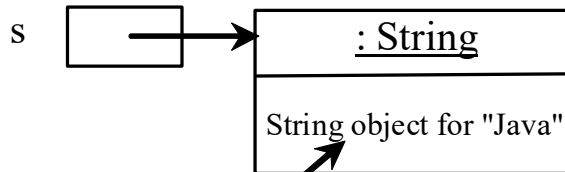


Trace Code

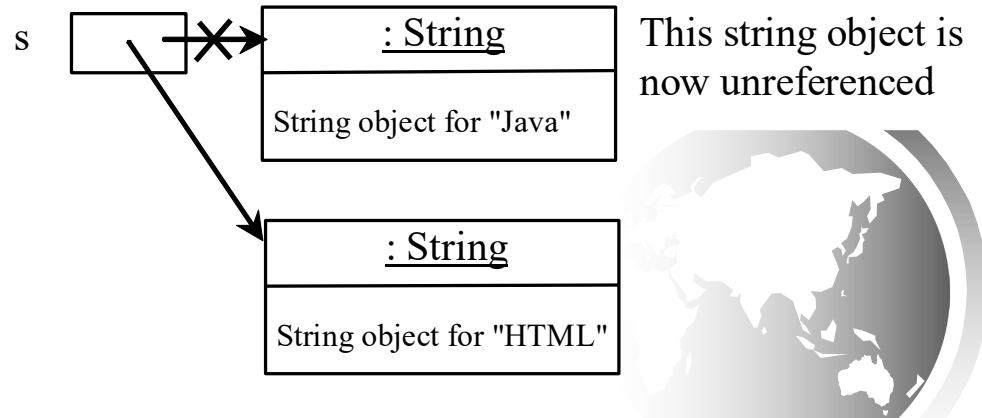
```
String s = "Java";
```

```
s = "HTML";
```

After executing `String s = "Java";`



After executing `s = "HTML";`



Interned Strings

Since strings are immutable and are frequently used, to improve efficiency and save memory, the JVM uses a unique instance for string literals with the same character sequence. Such an instance is called *interned*.

For example, the following statements:



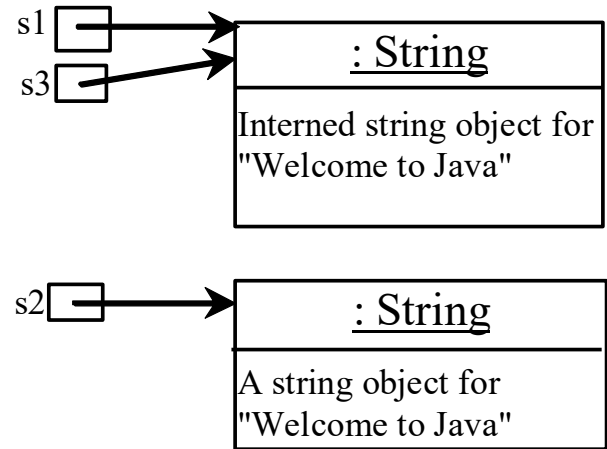
Examples

```
String s1 = "Welcome to Java";
```

```
String s2 = new String("Welcome to Java");
```

```
String s3 = "Welcome to Java";
```

```
System.out.println("s1 == s2 is " + (s1 == s2));  
System.out.println("s1 == s3 is " + (s1 == s3));
```



display

`s1 == s2` is false

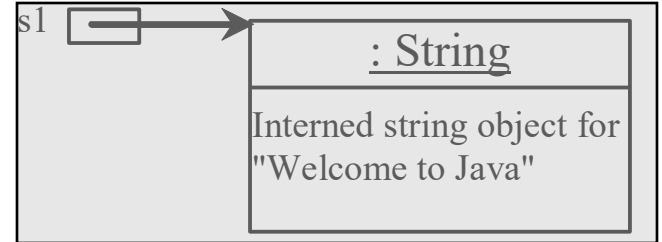
`s1 == s3` is true

A new object is created if you use the `new` operator.

If you use the string initializer, no new object is created if the interned object is already created.

Trace Code

```
String s1 = "Welcome to Java";  
String s2 = new String("Welcome to Java");  
String s3 = "Welcome to Java";
```

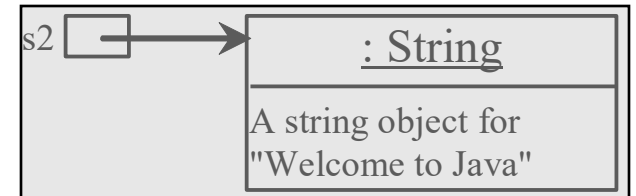
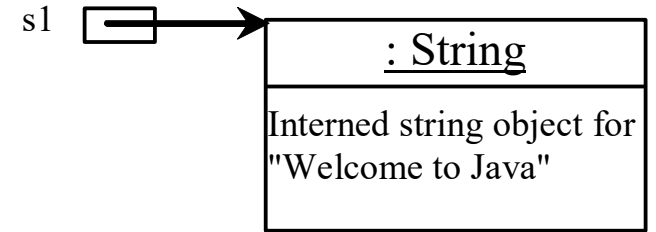


Trace Code

```
String s1 = "Welcome to Java";
```

```
String s2 = new String("Welcome to Java");
```

```
String s3 = "Welcome to Java";
```

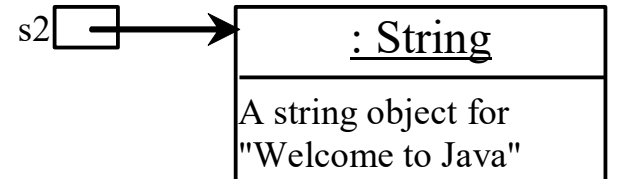
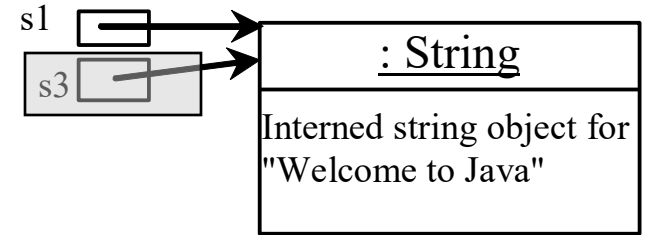


Trace Code

```
String s1 = "Welcome to Java";
```

```
String s2 = new String("Welcome to Java");
```

```
String s3 = "Welcome to Java";
```



Test Your Knowledge



String s1="test"

String s2= new String(s1)

s2==s1 ==> **True** / **False**

new String("test").equals("test") ==> **True** / **False**

new String("test") == "test" ==> **True** / **False**

new String("test") == new String("test") ==> **True** / **False**

"test" == "test" ==> **True** / **False**

"test" == "Test" ==> **True** / **False**

"test" == "te" + "st" ==> **True** / **False**



String s1="test"

String s2= new String(s1)

s2==s1 $\Rightarrow$ **False**

new String("test").equals("test") $\Rightarrow$ **True** (they have the same value)

new String("test") == "test" $\Rightarrow$ **False** (not the same object)

new String("test") == new String("test") $\Rightarrow$ **False** (not the same object)

"test" == "test" $\Rightarrow$ **True** (literals are interned by the compiler)

"test" == "Test" $\Rightarrow$ **False** (use equalsIgnoreCase())

"test" == "te" + "st" $\Rightarrow$ **True** (string literals are concatenated by JVM)

Question

// Java program to show how to compare Strings

```
public class Test {  
    public static void main(String[] args)  
    {  
        String s1 = "Ram";  
        String s2 = "Ram";  
        String s3 = new String("Ram");  
        String s4 = new String("Ram");  
        String s5 = "Shyam";  
        System.out.println("Comparing strings with equals:");  
        System.out.println(s1.equals(s2));  
        System.out.println(s1.equals(s3));  
        System.out.println(s1.equals(s5));  
        System.out.println("Comparing strings with ==:");  
        System.out.println(s1 == s2);  
        System.out.println(s1 == s3);  
        System.out.println(s3 == s4);  
    }  
}
```



Answer

Comparing strings with equals:

true

true

false

`equals()` is a method that compares the **values of two objects.**

Comparing strings with `==`:

true

false

false

`==` is an operator that checks **memory locations (address) of two objects.**

Question

To create the string Welcome to Java, you may use a statement like this:

```
String s = "Welcome to Java";
```

or:

```
String s = new String("Welcome to  
Java");
```

Which one is better? Why?



Answer

- ♦ `String s = "Welcome to Java";`
- ♦ is better, because this type of string is stored as an interned string. The interned strings of the same value share the same object.



Replacing and Splitting Strings

java.lang.String

+replace(oldChar: char,
newChar: char): String

Returns a new string that replaces **all matching character** in this string with the **new character**.

+replaceFirst(oldString: String,
newString: String): String

Returns a new string that replaces the **first matching** substring in this string with the new substring.

+replaceAll(oldString: String,
newString: String): String

Returns a new string that replace all matching substrings in this string with the **new substring**.

+split(delimiter: String):
String[]

Returns an array of strings consisting of the substrings **split by the delimiter**.



Replacing Strings ... cont.

- Once a string is created, its contents **cannot be changed**.
- The methods `replace`, `replaceFirst`, and `replaceAll` return a **new string** derived from the original string (**without changing the original string!**).
- Several versions of the `replace` methods are provided to replace a character or a substring in the string with a new character or a new substring.



Replacing Examples

"Welcome".replace('e', 'A') returns a new string, **WAIcomA**.

"Welcome".replace("e", "AB") returns a new string,
WABlcomAB.

"Welcome".replace("el", "AB") returns a new string,
WABcome.

"Welcome".replaceFirst("e", "AB") returns a new string,
WABlcome.

"Welcome".replaceAll("e", "AB") returns a new string,
WABlcomAB.

"Welcome to Java".replaceAll("to", "AAA") returns a new
string, **Welcome AAA Java**



Question

Suppose:

String s1 = "Welcome to Java";

What are the results of the following expressions?

- a) "Welcome to Java".replace("Java", "HTML")
- b) s1.replace('o', 'T')
- c) s1.replaceAll("o", "T")
- d) s1.replaceFirst("o", "T")



Answer

- a) Welcome to HTML
- b) WelcTme tT Java
- c) WelcTme tT Java
- d) WelcTme to Java



Splitting a String

- ◆ `String [ ] split (String regex, int limit)`
- ◆ **regex** - a delimiting regular expression
- ◆ **limit** - the result threshold
- ◆ **limit > 0** : If this is the case then the pattern will be applied **at most limit-1 times**, the resulting array's length will not be more than n, and the resulting array's last entry will contain all input beyond the last matched pattern.
- ◆ **limit < 0** : In this case, the pattern will be applied **as many times as possible**, and the resulting array can be of any size.
- ◆ **limit = 0** : In this case, the pattern will be applied **as many times as possible**, the resulting array can be of any size, and **trailing empty strings will be discarded**.



Splitting a String

- The split method can be used to **extract tokens** from a string with the **specified delimiters**.
- For example, the following code:

```
String[] tokens = "Java#HTML#Perl".split("#",0);  
for (int i = 0; i < tokens.length; i++)  
    System.out.print(tokens[i] + " ");
```

Displays:

Java HTML Perl



Splitting a String: Example

```
String[] tokens = "Java#HTML#Perl#".split("#", -1);  
for (int i = 0; i < tokens.length; i++)  
    System.out.print(tokens[i] + " ");  
System.out.print(tokens.length);
```

Java HTML Perl 4

```
String[] tokens = "Java#HTML#Perl#".split("#", 0);  
for (int i = 0; i < tokens.length; i++)  
    System.out.print(tokens[i] + " ");  
System.out.print(tokens.length);
```

Java HTML Perl 3



Question

Split:

“Welcome to Java and HTML”

into an array tokens delimited by a **space** and assign the first two tokens into s1 and s2.



Answer

```
♦ String[] tokens = "Welcome to  
  Java and HTML".split(' ');  
♦ String s1 = tokens[0];  
♦ String s2 = tokens[1];
```



Example:

```
String str = "1,2,3,,,";  
System.out.println ("Negative:");  
String[] arrOfStr = str.split(",", -1);  
for (int i = 0; i < arrOfStr.length; i++)  
    System.out.println(arrOfStr[i]);
```

```
System.out.println ("Zero:");  
String[] arrOfStr2 = str.split(",", 0);  
for (int i = 0; i < arrOfStr2.length; i++)  
    System.out.println(arrOfStr2[i]);
```

```
System.out.println ("Positive:");  
String[] arrOfStr3 = str.split(",", 2);  
for (int i = 0; i < arrOfStr3.length; i++)  
    System.out.println(arrOfStr3[i] + " ");
```



Answer

Negative:

1
2
3

Zero:

1
2
3

Positive:

1
2, 3, , ,



Matching, Replacing and Splitting by Patterns

- Often you will need to write code that validates user input, such as to check whether the input is a number, a string with all lowercase letters, or a Social Security number. How do you write this type of code? A simple and effective way to accomplish this task is to use the *regular expression*.
- *A regular expression* (abbreviated regex) is a string that describes a **pattern** for matching a set of strings.
- You can **match**, **replace**, or **split** a string by specifying a pattern.
- This is an extremely useful and powerful feature.



Matching, Replacing and Splitting by Patterns

Regular expression is complex to beginning students. For this reason, two simple patterns are used in this section. Please refer to Supplement III.F, “Regular Expressions,” for further studies.

```
"Java".matches("Java");  
"Java".equals("Java");
```

} evaluate to **true**.

However, the **matches** method is more powerful. It can match not only a fixed string, but also a set of strings that follow a pattern. For example, the following statements all evaluate to true:

```
"Java is fun".matches("Java.*");  
"Java is cool".matches("Java.*");
```



Matching, Replacing and Splitting by Patterns

`"Java is cool".matches("Java.*");`

Java.* is a regular expression. It describes a string pattern that begins with Java followed by any zero or more characters. Here, the substring matches any zero or more characters.

The following statement evaluates to true.

`"440-02-4534".matches("\\d{3}-\\d{2}-\\d{4}");`

Here `\\d` represents a single digit, and `\\d{3}` represents three digits.



Matching, Replacing and Splitting by Patterns

The `replaceAll`, `replaceFirst`, and `split` methods can be used with a regular expression. For example, the following statement returns a new string that replaces `$`, `+`, or `#` in `"a+b$#c"` by the string `NNN`.

```
String s = "a+b$#c".replaceAll("[$+#]", "NNN");  
System.out.println(s);
```

Here the regular expression `[$+#]` specifies a pattern that matches `$`, `+`, or `#`. So, the output is `aNNNbNNNNNNNc`.



Matching, Replacing and Splitting by Patterns

The following statement splits the string into an array of strings delimited by some punctuation marks.

```
String[] tokens = "Java,C?C#,C++".split("[.,:;?]);
```

```
for (int i = 0; i < tokens.length; i++)
```

```
    System.out.println(tokens[i]);
```

In this example, the regular expression `[.,:;?]` specifies a pattern that matches `.`, `,`, `:`, `;`, or `?`. Each of these characters is a delimiter for splitting the string. Thus, the string is split into `Java`, `C`, `C#`, and `C++`, which are stored in array `tokens`.

Regular Expression Syntax

Regular Expression	Matches	Example
<code>x</code>	a specified character <code>x</code>	<code>Java</code> matches <code>Java</code>
<code>.</code>	any single character	<code>Java</code> matches <code>J..a</code>
<code>(ab cd)</code>	<code>ab</code> or <code>cd</code>	<code>ten</code> matches <code>t(en im)</code>
<code>[abc]</code>	<code>a</code> , <code>b</code> , or <code>c</code>	<code>Java</code> matches <code>Ja[uvw]a</code>
<code>[^abc]</code>	any character except <code>a</code> , <code>b</code> , or <code>c</code>	<code>Java</code> matches <code>Ja[^ars]a</code>
<code>[a-z]</code>	<code>a</code> through <code>z</code>	<code>Java</code> matches <code>[A-M]av[a-d]</code>
<code>[^a-z]</code>	any character except <code>a</code> through <code>z</code>	<code>Java</code> matches <code>Jav[^b-d]</code>
<code>[a-e[m-p]]</code>	<code>a</code> through <code>e</code> or <code>m</code> through <code>p</code>	<code>Java</code> matches <code>[A-G[I-M]]av[a-d]</code>
<code>[a-e&&[c-p]]</code>	intersection of <code>a-e</code> with <code>c-p</code>	<code>Java</code> matches <code>[A-P&&[I-M]]av[a-d]</code>
<code>\d</code>	a digit, same as <code>[0-9]</code>	<code>Java2</code> matches <code>"Java[\d]"</code>
<code>\D</code>	a non-digit	<code>\$Java</code> matches <code>"[\D][\D]ava"</code>
<code>\w</code>	a word character	<code>Java1</code> matches <code>"[\w]ava[\w]"</code>
<code>\W</code>	a non-word character	<code>\$Java</code> matches <code>"[\W][\w]ava"</code>
<code>\s</code>	a whitespace character	<code>"Java 2"</code> matches <code>"Java\s2"</code>
<code>\S</code>	a non-whitespace char	<code>Java</code> matches <code>"[\S]ava"</code>
<code>p*</code>	zero or more occurrences of pattern <code>p</code>	<code>aaaabb</code> matches <code>"a*bb"</code> <code>ababab</code> matches <code>"(ab)*"</code>
<code>p+</code>	one or more occurrences of pattern <code>p</code>	<code>a</code> matches <code>"a+b*"</code> <code>able</code> matches <code>"(ab)+.*"</code>
<code>p?</code>	zero or one occurrence of pattern <code>p</code>	<code>Java</code> matches <code>"J?Java"</code> <code>Java</code> matches <code>"J?ava"</code>
<code>p{n}</code>	exactly <code>n</code> occurrences of pattern <code>p</code>	<code>Java</code> matches <code>"Ja{1}.*"</code> <code>Java</code> does not match <code>".{2}"</code>
<code>p{n,}</code>	at least <code>n</code> occurrences of pattern <code>p</code>	<code>aaaa</code> matches <code>"a{1,}"</code> <code>a</code> does not match <code>"a{2,}"</code>
<code>p{n,m}</code>	between <code>n</code> and <code>m</code> occurrences (inclusive)	<code>aaaa</code> matches <code>"a{1,9}"</code> <code>abb</code> does not match <code>"a{2,9}bb"</code>

Examples

```
String s = "Java Java Java".replaceAll("v\\w", "wi") ;
```

Jawi Jawi Jawi

```
String s = "Java Java Java".replaceFirst("v\\w", "wi") ;
```

Jawi Java Java

```
String[] s = "Java1HTML2Perl".split("\\d");
```

Java HTML Perl



Matching Strings

```
String Str = new String("Welcome to Tutorialspoint.com");
System.out.print("matches Return Value :" );
System.out.println(Str.matches("(.*)Tutorials(.*)"));
System.out.print("equals Return Value :" );
System.out.println(Str.equals("(.*)Tutorials(.*)"));
System.out.print("matches Return Value :" );
System.out.println(Str.matches("Tutorials"));
System.out.print("equals Return Value :" );
System.out.println(Str.equals("Tutorials"));
System.out.print("matches Return Value :" );
System.out.println(Str.matches("Welcome(.*)"));
System.out.print("equals Return Value :" );
System.out.println(Str.equals("Welcome(.*)"));
```

matches Return Value :true
equals Return Value :false
matches Return Value :false
equals Return Value :false
matches Return Value :true
equals Return Value :false



Test Your Knowledge



```
System.out.println("as".matches(".s"));
System.out.println("mk".matches(".s"));
System.out.println("mst".matches(".s"));
System.out.println("amms".matches(".s"));
System.out.println("mas".matches("..s"));
System.out.println("abc".matches("\\d"));
System.out.println("1".matches("\\d"));
System.out.println("4443".matches("\\d"));
System.out.println("323abc".matches("\\d"));
System.out.println("abc".matches("\\D"));
System.out.println("1".matches("\\D"));
System.out.println("4443abc".matches("\\D"));
System.out.println("mak123".matches("\\D*\\d*"));
```



Test Your Knowledge



<code>System.out.println("as".matches(".s"));</code>	true
<code>System.out.println("mk".matches(".s"));</code>	false
<code>System.out.println("mst".matches(".s"));</code>	false
<code>System.out.println("amms".matches(".s"));</code>	false
<code>System.out.println("mas".matches("..s"));</code>	true
<code>System.out.println("abc".matches("\\d"));</code>	false
<code>System.out.println("1".matches("\\d"));</code>	true
<code>System.out.println("4443".matches("\\d"));</code>	false
<code>System.out.println("323abc".matches("\\d"));</code>	false
<code>System.out.println("abc".matches("\\D"));</code>	false
<code>System.out.println("1".matches("\\D"));</code>	false
<code>System.out.println("4443abc".matches("\\D"));</code>	false
<code>System.out.println("mak123".matches("\\D*\\d*"));</code>	true

Question

- ♦ What is the output of the following code?
- ♦ `String s1 = "Welcome to Java";`
- ♦ `String s2 = s1.replace("o", "abc");`
- ♦ `System.out.println(s1);`
- ♦ `System.out.println(s2);`



Answer

- ◆ The output is
- ◆ Welcome to Java
- ◆ Welcabcme tabc Java
- ◆ Hint: No method in the String class can change the content of the string. String is an immutable class.



Question

Let s1 be " Welcome " and s2 be " welcome ".
Write the code for the following statements:
Replace all occurrences of the character e
with E in s1 and assign the new string to s3.



Answer

```
String s3 = s1.replace('e', 'E');
```



Question

- ◆ Does any method in the String class change the contents of the string?



Answer

◆ No



Question

- ✦ Suppose string `s` is created using `new String()`; what is `s.length()`?



Answer

✦ 0



Question

- ◆ Show the output of the following code.
- ◆ `public class Test {`
- ◆ `public static void main(String[] args) {`
- ◆ `System.out.println("Hi, ABC, good".matches("ABC "));`
- ◆ `System.out.println("Hi, ABC, good".matches(".*ABC.*"));`
- ◆ `System.out.println("A,B;C".replaceAll(";", "#"));`
- ◆ `System.out.println("A,B;C".replaceAll("[,;]", "#"));`
- ◆ `String[] tokens = "A,B;C".split("[,;]");`
- ◆ `for (int i = 0; i < tokens.length; i++)`
- ◆ `System.out.print(tokens[i] + " ");`
- ◆ `}`
- ◆ `}`



Answer

- ♦ false
- ♦ true
- ♦ A,B;C
- ♦ A#B#C
- ♦ A B C



Convert Character and Numbers to Strings

- ◆ The String class provides several static **valueOf** methods for converting a **character**, an **array of characters**, and **numeric values** to strings.
- ◆ These methods have the same name **valueOf** with different argument types **char**, **char[]**, **double**, **long**, **int**, and **float**.
- ◆ For example, to convert a double value to a string, use:

```
String s = String.valueOf(5.44);
```

The return value is string consists of characters:
'5', '.', '4', and '4'.

StringBuilder and StringBuffer

- ◆ The **StringBuilder** and **StringBuffer** classes are alternatives to the **String** class.
- ◆ In general, a **StringBuilder/StringBuffer** can be used wherever a string is used.
- ◆ **StringBuilder** and **StringBuffer** are more flexible than **String**.
- ◆ You can add, insert, or append new contents into a string buffer or builder, whereas the value of a String object is fixed once the string is created.

StringBuilder and StringBuffer

- ◆ The **StringBuilder** class is similar to the **StringBuffer** except that the **methods for modifying the buffer in StringBuffer are synchronized**, which means that only one task is allowed to execute the methods.
- ◆ When to use **StringBuffer**?
Use **StringBuffer** if the class might be accessed by **multiple tasks concurrently**, because synchronization is needed in this case to prevent corruptions to **StringBuffer**.

StringBuilder Constructors

- `StringBuilder x = new StringBuilder ();`
- `StringBuilder x = new StringBuilder (10);`
- `StringBuilder x = new StringBuilder (“CPCS203”);`

java.lang.StringBuilder
<code>+StringBuilder()</code>
<code>+StringBuilder(capacity: int)</code>
<code>+StringBuilder(s: String)</code>

Constructs an empty string builder with capacity 16.

Constructs a string builder with the specified capacity.

Constructs a string builder with the specified string.

Note that when constructing a string builder using a specified string, then the capacity will be:

the length of the string + the default capacity (16)

Modifying Strings in the Builder

java.lang.StringBuilder

+append(data: char[]): StringBuilder
+append(data: char[], offset: int, len: int): StringBuilder
+append(v: *aPrimitiveType*): StringBuilder
+append(s: String): StringBuilder

Appends a char array into this string builder.

Appends a subarray in data into this string builder.

Appends a primitive type value as a string to this builder.

Appends a string to this string builder.

```
StringBuilder sbf = new StringBuilder("We are geeks ");  
char[] astr = new char[] {'G', 'E', 'E', 'k', 'S'};  
sbf.append(astr);  
System.out.println(sbf);
```



Modifying Strings in the Builder

java.lang.StringBuilder

+append(data: char[]): StringBuilder
+append(data: char[], offset: int, len: int): StringBuilder
+append(v: *aPrimitiveType*): StringBuilder
+append(s: String): StringBuilder

Appends a char array into this string builder.

Appends a subarray in data into this string builder.

Appends a primitive type value as a string to this builder.

Appends a string to this string builder.

```
StringBuilder sbf = new StringBuilder("We are geeks ");  
char[] astr = new char[] {'G', 'E', 'E', 'k', 'S'};  
sbf.append(astr);  
System.out.println(sbf);
```

We are geeks GEEkS



Modifying Strings in the Builder

java.lang.StringBuilder

+append(data: char[]): StringBuilder
+append(data: char[], offset: int, len: int): StringBuilder
+append(v: *aPrimitiveType*): StringBuilder
+append(s: String): StringBuilder

Appends a char array into this string builder.

Appends a subarray in data into this string builder.

Appends a primitive type value as a string to this builder.

Appends a string to this string builder.

offset - the index of the first char in the subarray to be inserted.

len - the number of chars in the subarray to be inserted.

```
StringBuilder sb = new StringBuilder("Geeks");  
char[] cstr = new char[] {'f', 'o', 'r', 'G', 'e', 'e', 'k', 's', 'q', 'q'};  
sb.append(cstr, 0, 8);  
System.out.println(sb);
```



Modifying Strings in the Builder

java.lang.StringBuilder

+append(data: char[]): StringBuilder
+append(data: char[], offset: int, len: int): StringBuilder
+append(v: *aPrimitiveType*): StringBuilder
+append(s: String): StringBuilder

Appends a char array into this string builder.

Appends a subarray in data into this string builder.

Appends a primitive type value as a string to this builder.

Appends a string to this string builder.

offset - the index of the first char in subarray to be inserted.

len - the number of chars in the subarray to be inserted.

```
StringBuilder sb = new StringBuilder("Geeks");  
char[] cstr = new char[] {'f', 'o', 'r', 'G', 'e', 'e', 'k', 's', 'q', 'q'};  
sb.append(cstr, 0, 8);  
System.out.println(sb);
```

GeeksforGeeks



Modifying Strings in the Builder

java.lang.StringBuilder

+append(data: char[]): StringBuilder
+append(data: char[], offset: int, len: int): StringBuilder
+append(v: aPrimitiveType): StringBuilder
+append(s: String): StringBuilder

Appends a char array into this string builder.

Appends a subarray in data into this string builder.

Appends a primitive type value as a string to this builder.

Appends a string to this string builder.

```
StringBuilder sbf = new StringBuilder("Welcome geeks!");  
sbf.append('T');  
System.out.println(sbf);
```



Modifying Strings in the Builder

java.lang.StringBuilder

+append(data: char[]): StringBuilder
+append(data: char[], offset: int, len: int): StringBuilder
+append(v: aPrimitiveType): StringBuilder
+append(s: String): StringBuilder

Appends a char array into this string builder.

Appends a subarray in data into this string builder.

Appends a primitive type value as a string to this builder.

Appends a string to this string builder.

```
StringBuilder sbf = new StringBuilder("Welcome geeks!");  
sbf.append('T');  
System.out.println(sbf);
```

Welcome geeks!T



Modifying Strings in the Builder

java.lang.StringBuilder

+append(data: char[]): StringBuilder
+append(data: char[], offset: int, len: int): StringBuilder
+append(v: *aPrimitiveType*): StringBuilder
+append(s: String): StringBuilder
+append(s: String, startIndex: int, endIndex: int)

Appends a char array into this string builder.

Appends a subarray in data into this string builder.

Appends a primitive type value as a string to this builder.

Appends a string to this string builder.

Append a substring into this string builder starting at startIndex up to the (exclusive) endIndex

```
StringBuilder x = new StringBuilder ("Hello World");  
System.out.println(x.append(" Java", 0, 5));
```

```
StringBuilder x2 = new StringBuilder ("Hello World");  
System.out.println(x2.append(" Java", 1, 3));
```



Modifying Strings in the Builder

java.lang.StringBuilder

+append(data: char[]): StringBuilder
+append(data: char[], offset: int, len: int): StringBuilder
+append(v: *aPrimitiveType*): StringBuilder
+append(s: String): StringBuilder
+append(s: String, startIndex: int, endIndex: int)

Appends a char array into this string builder.

Appends a subarray in data into this string builder.

Appends a primitive type value as a string to this builder.

Appends a string to this string builder.

Append a substring into this string builder starting at startIndex up to the (exclusive) endIndex

```
StringBuilder x = new StringBuilder ("Hello World");  
System.out.println(x.append(" Java", 0, 5));
```

>> Hello World Java

```
StringBuilder x2 = new StringBuilder ("Hello World");  
System.out.println(x2.append(" Java", 1, 3));
```

>>Hello WorldJa



Modifying Strings in the Builder

java.lang.StringBuilder

+delete(startIndex: int, endIndex: int): StringBuilder

+deleteCharAt(index: int): StringBuilder

+insert(index: int, data: char[], offset: int, len: int): StringBuilder

+insert(offset: int, data: char[]): StringBuilder

+insert(offset: int, b: *aPrimitiveType*): StringBuilder

+insert(offset: int, s: String): StringBuilder

Deletes characters from startIndex to endIndex.

Deletes a character at the specified index.

Inserts a subarray of the data in the array to the builder at the specified index.

Inserts data into this builder at the position offset.

Inserts a value converted to a string into this builder.

Inserts a string into this builder at the position offset.

```
StringBuilder x = new StringBuilder ("Hello World");  
System.out.println(x.delete(5, 8));
```

Hellold

```
StringBuilder x = new StringBuilder ("Hello World");  
System.out.println(x.deleteCharAt(5));
```

HelloWorld



Modifying Strings in the Builder

java.lang.StringBuilder

+delete(startIndex: int, endIndex: int): StringBuilder

Deletes characters from startIndex to endIndex.

+deleteCharAt(index: int): StringBuilder

Deletes a character at the specified index.

+insert(index: int, data: char[], offset: int, len: int): StringBuilder

Inserts a subarray of the data in the array to the builder at the specified index.

+insert(offset: int, data: char[]): StringBuilder

Inserts data into this builder at the position offset.

+insert(offset: int, b: *aPrimitiveType*): StringBuilder

Inserts a value converted to a string into this builder.

+insert(offset: int, s: String): StringBuilder

Inserts a string into this builder at the position offset.

+insert(offset: int, s: String, startIndex: int, endIndex: int)

Insert a substring into this builder at the position offset at specified index.

index - position at which to insert subarray.

offset - the index of the first char in subarray to be inserted.

len - the number of chars in the subarray to be inserted.

```
StringBuilder sb1 = new StringBuilder("javapoint");
```

```
char[] str1 = {'.', 'c', 'o', 'm'};
```

```
sb1.insert(sb1.length(), str1, 0, 4);
```

```
System.out.println("after insert = " + sb1);
```

Javapoint.com



Modifying Strings in the Builder

java.lang.StringBuilder

+delete(startIndex: int, endIndex: int): StringBuilder

Deletes characters from startIndex to endIndex.

+deleteCharAt(index: int): StringBuilder

Deletes a character at the specified index.

+insert(index: int, data: char[], offset: int, len: int): StringBuilder

Inserts a subarray of the data in the array to the builder at the specified index.

+insert(offset: int, data: char[]): StringBuilder

Inserts data into this builder at the position offset.

+insert(offset: int, b: *aPrimitiveType*): StringBuilder

Inserts a value converted to a string into this builder.

+insert(offset: int, s: String): StringBuilder

Inserts a string into this builder at the position offset.

+insert(offset: int, s: String, startIndex: int, endIndex: int)

Insert a substring into this builder at the position offset at specified index.

```
StringBuilder x = new StringBuilder ("Hello World");  
System.out.println(x.insert(5,"HTML and Java",8,13));
```

```
StringBuilder x = new StringBuilder ("Hello World");  
System.out.println(x.insert(5,"HTML and Java"));
```

Hello Java World

HelloHTML and Java World

Modifying Strings in the Builder

java.lang.StringBuilder

```
+replace(startIndex: int, endIndex: int, s: String): StringBuilder  
+reverse(): StringBuilder  
+setCharAt(index: int, ch: char): void
```

Replaces the characters in this builder from `startIndex` to `endIndex` with the specified string.

Reverses the characters in the builder.

Sets a new character at the specified index in this builder.

```
StringBuilder x = new StringBuilder ("Hello World");
```

```
x.setCharAt(3, 'e');
```

```
System.out.println(x);
```

Heleo World

```
StringBuilder x1 = x.reverse();
```

```
System.out.println(x1);
```

dlroW olleH



Question

What is the output of the following ?

```
StringBuilder sb = new StringBuilder();  
System.out.println(sb.append("Welcome"));  
System.out.println(sb.append(" to Java"));  
System.out.println(sb.insert(11, "HTML and "));  
System.out.println(sb.delete(8, 11));  
System.out.println(sb.deleteCharAt(8));  
System.out.println(sb.reverse());  
System.out.println(sb.replace(11, 15, "HTML"));  
sb.setCharAt(0, 'w');  
System.out.println(sb);
```



Answer

Welcome

Welcome to Java

Welcome to HTML and Java

Welcome HTML and Java

Welcome TML and Java

avaJ dna LMT emocleW

avaJ dna LMHTMLocleW

wvaJ dna LMHTMLocleW



Test Your Knowledge

```
StringBuilder x = new StringBuilder ("welcome");  
System.out.println(x.append("Welcome "));  
System.out.println(x.append("to Java"));  
System.out.println(x.insert(7, " HTML and "));  
System.out.println(x.delete(16, 25));  
System.out.println(x.deleteCharAt(16));  
System.out.println(x.deleteCharAt(16));  
System.out.println(x.reverse());  
System.out.println(x.replace(9, 13, "nohtyP"));  
x.setCharAt(x.length()-1, 'W');  
System.out.println(x);  
System.out.println(x.reverse());
```



Test Your Knowledge

welcomeWelcome

welcomeWelcome to Java

welcome HTML and Welcome to Java

welcome HTML andto Java

welcome HTML ando Java

welcome HTML and Java

avaJ dna LMTH emoclew

avaJ dna nohtyP emoclew

avaJ dna nohtyP emocleW

Welcome Python and Java



The toString, capacity, length, setLength, and charAt Methods

java.lang.StringBuilder	
+toString(): String	Returns a string object from the string builder.
+capacity(): int	Returns the capacity of this string builder.
+charAt(index: int): char	Returns the character at the specified index.
+length(): int	Returns the number of characters in this builder.
+setLength(newLength: int): void	Sets a new length in this builder.
+substring(startIndex: int): String	Returns a substring starting at startIndex.
+substring(startIndex: int, endIndex: int): String	Returns a substring from startIndex to endIndex-1.
+trimToSize(): void	Reduces the storage size used for the string builder.

An empty StringBuffer class contains the default **16** character capacity. If the number of the character increases from its current capacity, it increases the capacity by **(oldcapacity) + new string length**.

```
StringBuilder x = new StringBuilder("Welcome to Java");  
System.out.println(x.capacity());  
System.out.println(x.length());  
x.setLength(8);  
System.out.println(x);  
System.out.println(x.capacity());  
System.out.println(x.length());  
StringBuilder y = new StringBuilder ("welcome");  
System.out.println(y.length());  
System.out.println(y.capacity());  
y.trimToSize();  
System.out.println(y.capacity());
```



Problem: Checking Palindromes Ignoring Non-alphanumeric Characters

This example gives a program that counts the number of occurrence of each letter in a string.

Assume the letters are not case-sensitive.

Palindromes are words that read the same backward and forward (Mom, Level, Radar, Eye)



PalindromeIgnoreNonAlphanumeric

Run

```

import java.util.Scanner;
public class PalindromelgnoreNonAlphanumeric
{
    /** Main method */
    public static void main(String[] args) {
        // Create a Scanner
        Scanner input = new Scanner(System.in);
        // Prompt the user to enter a string
        System.out.print("Enter a string: ");
        String s = input.nextLine();
        // Display result
        System.out.println("Ignoring non-alphanumeric characters, \nis " + s + " a palindrome?
                           " + isPalindrome(s));
    }
    /** Return true if a string is a palindrome */
    public static boolean isPalindrome(String s) {
        // Create a new string by eliminating non-alphanumeric chars
        String s1 = filter(s);
        // Create a new string that is the reversal of s1
        String s2 = reverse(s1);
        // Compare if the reversal is the same as the original string
        return s2.equals(s1);
    }
}

```

//*****



```

/** Create a new string by eliminating non-alphanumeric chars */
public static String filter(String s) {
    // Create a string builder
    StringBuilder stringBuilder = new StringBuilder();
    // Examine each char in the string to skip alphanumeric char
    for (int i = 0; i < s.length(); i++) {
        if (Character.isLetterOrDigit(s.charAt(i)))
            { stringBuilder.append(s.charAt(i)); }
    }
    // Return a new filtered string
    return stringBuilder.toString();
}
//*****

```

```

/** Create a new string by reversing a specified string */
public static String reverse(String s) {
    StringBuilder stringBuilder = new StringBuilder(s);
    stringBuilder.reverse();
    // Invoke reverse in StringBuilder
    return stringBuilder.toString();
}
}

```



Question

- ✦ Show the output of the following program:
- ✦ `public class Test {`
- ✦ `public static void main(String[] args) {`
- ✦ `String s = "Java";`
- ✦ `StringBuilder builder = new StringBuilder(s);`
- ✦ `change(s, builder);`
- ✦ `System.out.println(s);`
- ✦ `System.out.println(builder);`
- ✦ `}`
- ✦ `private static void change(String s, StringBuilder builder) {`
- ✦ `s = s + " and HTML";`
- ✦ `builder.append(" and HTML");`
- ✦ `}`
- ✦ `}`



Answer

- ♦ The output is
- ♦ Java
- ♦ Java and HTML
- ♦
- ♦ NOTE: Inside the method, the statement `s = s + " and HTML"` creates a new String object `s`, which is different from the original String object passed to the `change(s, builder)` method. The original String object has not been changed. Therefore, the output from the original string is Java. Inside the method, the content of the StringBuilder object is changed to Java and HTML. Therefore, the output from builder is Java and HTML.



Question

- ◆ What is the difference between `StringBuilder` and `StringBuffer`?



Answer

- ♦ The `StringBuilder` class, introduced in JDK 1.5, is similar to `StringBuffer` except that the update methods in `StringBuffer` are synchronized.



Question

- ◆ How do you create a string builder from a string? How do you return a string from a string builder?



Answer

- ◆ Use the StringBuilder's constructor to create a string buffer for a string and use the toString method in StringBuilder class to return a string from a StringBuilder.



Question

- ✦ Write three statements to reverse a string `s` using the `reverse` method in the `StringBuilder` class.



Answer

```
StringBuilder sb = new StringBuilder(s);  
sb.reverse();  
s = sb.toString();
```



Question

- ✦ Write three statements to delete a substring from a string `s` of 20 characters, starting at index 4 and ending with index 10. Use the `delete` method in the `StringBuilder` class.



Answer

- ◆ `StringBuilder sb = new StringBuilder(s);`
`sb.delete(4, 11);`
- ◆ `s = sb.toString();`



Question

- ◆ What is the internal storage for characters in a string and a string builder?



Answer

- ◆ Both string and string buffer use arrays to hold characters. The array in a string is fixed once a string is created. The array in a string buffer may change if the buffer capacity is changed. To accommodate the change, a new array is created.



Question

- ✦ Suppose that s1 and s2 are given as follows:
- ✦ `StringBuilder s1 = new StringBuilder("Java");`
- ✦ `StringBuilder s2 = new StringBuilder("HTML");`
- ✦ Show the value of s1 after each of the following statements. Assume that the statements are independent.
- ✦ a. `s1.append(" is fun");`
- ✦ b. `s1.append(s2);`
- ✦ c. `s1.insert(2, "is fun");`
- ✦ d. `s1.insert(1, s2);`
- ✦ e. `s1.charAt(2);`
- ✦ f. `s1.length();`
- ✦ g. `s1.deleteCharAt(3);`
- ✦ h. `s1.delete(1, 3);`
- ✦ i. `s1.reverse();`
- ✦ j. `s1.replace(1, 3, "Computer");`
- ✦ k. `s1.substring(1, 3);`
- ✦ l. `s1.substring(2);`



Answer

- ✦ (a) Java is fun
- ✦ (b) JavaHTML
- ✦ (c) Jais funva
- ✦ (d) JHTMLava
- ✦ (e) v
- ✦ (f) 4
- ✦ (g) Jav
- ✦ (h) Ja
- ✦ (i) avaJ
- ✦ (j) JComputera
- ✦ (k) av
- ✦ (l) va

